Linux Learning – Day 1 Complete Notes

1. Understanding Our Linux Environment

We are learning Linux using:

- Windows Subsystem for Linux (WSL2)
- Ubuntu 24.04.4
- Bash terminal

What is WSL?

WSL stands for Windows Subsystem for Linux.

It allows us to run a Linux environment directly inside Windows without installing a separate virtual machine in the usual way.

Commands used to check WSL

Check installed Linux distributions:

```
None
wsl -l -v
```

Meaning:

- `wsl`: Windows Subsystem for Linux.
- `-l`: List installed distributions.
- `-v`: Show their WSL version.

Open Ubuntu as root:

```
None
wsl -d Ubuntu -u root
```

- `-d`: Specifies the distribution.
- `-u`: Specifies the Linux user.
- `root`: Opens the terminal as the root user.

Important Linux users

Linux has different types of users:

1. Root user: The administrator with extensive system privileges.
2. Normal user: A regular account with its own home directory and permissions.

We created a user:

```
None
sudo adduser vishal
```

Added the user to the sudo group:

```
None
sudo usermod -aG sudo vishal
```

Switched to that user:

```
None
su - vishal
```

The `-` starts a login shell and loads the user's login environment.

---

2. Basic Linux Navigation Commands

These commands help us understand where we are and navigate through the Linux filesystem.

2.1 pwd

```
None
pwd
```

Full form: Print Working Directory.

It tells us our current location.

Example:

```
None
/home/vishal
```

This means we are inside the home directory of user Vishal.

2.2 whoami

```
None
whoami
```

Displays the username of the current user.

Example:

```
None
vishal
```

2.3 users

```
None
users
```

Displays usernames currently logged into the system.

Important difference:

- `whoami`: Shows the user executing the command.
- `users`: Shows users with active login sessions.

2.4 cd

Full form: Change Directory.

Used to move between directories.

```
None
cd folder_name
```

Examples:

```
None
cd Documents
```

Moves into Documents.

```
None
cd ..
```

Moves one level up to the parent directory.

```
None
cd ~
```

Moves to the home directory of the current user.

```
None
cd /
```

Moves to the root directory of the entire Linux filesystem.

Important symbols

Symbol	Meaning
`/`	Root of the entire filesystem
`~`	Current user's home directory
`.`	Current directory
`..`	Parent directory

Example:

If current location is:

```
None
/home/vishal/projects/backend
```

Then:

```
None
cd ..
```

Takes us to:

```
None
/home/vishal/projects
```

It does NOT automatically take us to `/home/vishal`.

---

3. Listing Files and Directories – ls

`ls` displays files and directories.

Basic command

```
None
ls
```

Important flags

1. -l : Long listing format

```
None
ls -l
```

Displays:

- Permissions
- Number of hard links
- Owner
- Group
- File size
- Modification date
- Filename

2. -a : Show hidden files

```
None
ls -a
```

Linux hidden files generally begin with a dot.

Examples:

```
None
.bashrc
.profile
.git
```

3. -la : Combine flags

```
None
ls -la
```

Shows hidden files in long listing format.

4. -R : Recursive listing

```
None
ls -R
```

Displays files and subdirectories recursively.

5. -lR : Long recursive listing

```
None
ls -lR
```

Displays detailed information recursively.

Important distinction

```
None
ls -la
```

Does NOT recursively display all subdirectories.

For recursive listing, use:

```
None
ls -laR
```

Understanding ls -l output

Example:

```
None
-rw-rw-r-- 1 vishal developers 108 Nov 11 17:48 backend.txt
```

Breakdown:

Part	Meaning
`-`	Regular file
`rw-rw-r--`	Permissions
`1`	Number of hard links
`vishal`	Owner
`developers`	Group
`108`	File size in bytes
`Nov 11 17:48`	Modification time
`backend.txt`	Filename

For directories, the first character is `d`.

Example:

```
None
drwxrws---
```

---

4. Linux File and Directory Operations

4.1 mkdir

Full form: Make Directory.

Creates a directory.

```
None
mkdir projects
```

Creates a directory named projects.

Create nested directories:

```
None
mkdir -p projects/backend/api
```

`-p` creates missing parent directories as required.

4.2 touch

Creates an empty file if it does not exist.

```
None
touch file1.txt
```

Important: If the file already exists, `touch` updates its timestamps instead of deleting its content.

4.3 cat

Displays file content.

```
None
cat file1.txt
```

You can also display multiple files:

```
None
cat file1.txt file2.txt
```

4.4 echo

Displays text in the terminal.

```
None
echo "Hello Linux"
```

Writing into a file using >

```
None
echo "I am learning Linux" > file1.txt
```

`>` redirects output into the file.

If the file already contains data, it overwrites the existing content.

Appending using >>

```
None
echo "Second line" >> file1.txt
```

`>>` appends text to the existing content without removing previous content.

Important mistake we encountered

Incorrect:

```
None
echo "I am linux" file1.txt
```

This simply prints both arguments in the terminal. It does not write into the file.

Correct:

```
None
echo "I am linux" > file1.txt
```

4.5 cp

Full form: Copy.

```
None
cp file1.txt file2.txt
```

Copies file1.txt into file2.txt.

Important:

- Original file remains.
- New copy is created.
- If destination already exists as a regular file, its content is overwritten.

Useful flags:

```
None
cp -r folder1 folder2
```

`-r`: Recursively copies directories and their contents.

4.6 mv

Full form: Move.

Move a file:

```
None
mv file1.txt Documents/
```

Rename a file:

```
None
mv old.txt new.txt
```

Important:

- `mv` can move files.
- `mv` can rename files.
- The original source location no longer contains the moved file.

4.7 rm

Full form: Remove.

```
None
rm file1.txt
```

Deletes a file.

Important flags:

```
None
rm -r folder1
```

`-r`: Recursively removes a directory and its contents.

```
None
rm -f file1.txt
```

`-f`: Force removal without prompting for confirmation in normal cases.

```
None
rm -rf folder1
```

Combines recursive and force removal.

WARNING: Be extremely careful with `rm -rf`, especially when using absolute paths or commands involving variables.

4.8 rmdir

Removes an empty directory.

```
None
rmdir folder1
```

It cannot remove a non-empty directory.

Difference:

Command	Purpose
`rm file.txt`	Deletes a file

`rmdir folder`	Deletes an empty directory
`rm -r folder`	Deletes a directory recursively

---

5. Understanding Linux Permissions

This is one of the most important topics we learned.

Linux permissions determine who can read, modify or execute a file and who can access a directory.

There are three permission categories:

1. Owner (u)
2. Group (g)
3. Others (o)

And three basic permissions:

Permission	Symbol	Numeric value
Read	r	4
Write	w	2
Execute	x	1
No permission	-	0

5.1 Understanding permission notation

Example:

```
None
-rwxr-xr--
```

Breakdown:

```
None
- | rwx | r-x | r--
```

```
    u     g     o
```

- First `-`: Regular file.
- `rwx`: Owner has read, write and execute.
- `r-x`: Group has read and execute.
- `r--`: Others have read only.

For directories, the first character is `d`.

5.2 Numeric permissions

We calculate permission values by adding the numeric values of the permissions.

Examples:

```
rwx = 4 + 2 + 1 = 7
rw- = 4 + 2 = 6
r-x = 4 + 1 = 5
r-- = 4
-wx = 2 + 1 = 3
--- = 0
```

Common permission combinations

Numeric	Symbolic	Meaning
777	rwxrwxrwx	Everyone has full access
755	rwxr-xr-x	Owner full, others read and execute
700	rwx------	Only owner has full access
666	rw-rw-rw-	Everyone can read and write
644	rw-r--r--	Owner read/write, others read
600	rw-------	Only owner can read/write
640	rw-r-----	Owner read/write, group read only

Remember:

The first digit represents owner, the second group, and the third others.

Example:

```
None
chmod 755 script.sh
```

Means:

- Owner = 7 = rwx
- Group = 5 = r-x
- Others = 5 = r-x

---

6. chmod – Changing Permissions

Full form: Change Mode.

Used to modify file and directory permissions.

6.1 Numeric method

```
None
chmod 755 file.txt
```

Changes permissions to:

```
None
-rwxr-xr-x
```

Other examples:

```
None
chmod 644 file.txt
chmod 600 file.txt
chmod 700 folder
chmod 777 file.txt
```

Avoid using 777 unnecessarily because it gives everyone full permissions.

6.2 Symbolic method

Instead of numeric values, we can use symbols.

Categories:

Symbol	Meaning
u	User/Owner
g	Group
o	Others
a	All

Operators:

Operator	Meaning
+	Add permission
-	Remove permission
=	Set exact permissions

Examples:

Add execute permission to owner:

```
None
chmod u+x file.txt
```

Add write permission to group:

```
None
chmod g+w file.txt
```

Remove read permission from others:

```
None
chmod o-r file.txt
```

Add execute permission to everyone:

```
None
chmod a+x file.txt
```

Remove all group permissions:

```
None
chmod g-rwx file.txt
```

Add read and write to group:

```
None
chmod g+rw file.txt
```

Multiple changes in one command

```
None
chmod g+rw,o-wx,o+r file.txt
```

Commas separate multiple permission changes.

Important mistake:

Incorrect:

```
None
chmod g+rw o-wx file.txt
```

Correct:

```
None
chmod g+rw,o-wx file.txt
```

6.3 Important practical we performed

We experimented with:

```
None
chmod 777 test.txt
chmod o-r test.txt
```

```
chmod g-rwx test.txt
chmod g+rw test.txt
chmod u-rwx test.txt
chmod 700 test.txt
```

When we removed the owner's read permission, `cat` failed.

When we removed the owner's write permission, appending content also failed.

Then we restored permissions using:

```
None
chmod 700 test.txt
```

And reading and writing worked again.

Important learning

Even if a file has no read or write permissions for its owner, the owner can generally still change its permissions using chmod.

Permission to change a file's mode is different from permission to read or write its contents.

---

7. Directory Permissions – Very Important

Directory permissions work differently from file permissions.

For a regular file:

Permission	Meaning
r	Read file contents
w	Modify file contents
x	Execute the file

For a directory:

Permission	Meaning
r	List directory entries
w	Create, delete or rename entries, subject to other checks
x	Enter/traverse/search the directory

Example

```
None
chmod 700 private-folder
```

Only the owner gets read, write and execute permissions.

Other users cannot normally enter that directory.

Why does directory x matter?

Suppose the path is:

```
None
/home/vishal/shared-project/backend.txt
```

A user needs execute/traverse permission on the relevant parent directories to access that file through this path.

Having permission on the final file alone is not sufficient.

Our practical problem

Initially, we created:

```
None
/home/vishal/shared-project
```

We gave it:

```
None
chmod 770 shared-project
```

And ownership:

```
None
sudo chown vishal:developers shared-project
```

But Rahul could not enter it.

Why?

Because `/home/vishal` did not allow Rahul to traverse it.

We fixed this using:

```
None
sudo chmod o+x /home/vishal
```

This gave others execute/traverse permission on the parent directory without granting directory listing permission.

Important: Directory `x` allows traversal/search. It does not automatically grant permission to list its contents.

---

8. Linux Users and Groups

Linux supports multiple users and groups.

A group allows us to give the same access permissions to multiple users.

Example:

We created two users:

- vishal
- rahul

And one group:

- developers

Both users became members of developers.

8.1 adduser

Creates a user account.

```
None
sudo adduser rahul
```

It interactively asks for password and other account information.

8.2 groupadd

Creates a new group.

```
None
sudo groupadd developers
```

8.3 usermod

Modifies an existing user account.

Add a supplementary group

```
None
sudo usermod -aG developers vishal
```

Add Rahul:

```
None
sudo usermod -aG developers rahul
```

Important flags:

Flag	Meaning
-a	Append to existing supplementary groups
-G	Specify supplementary groups
-g	Change primary group

Important difference

```
None
sudo usermod -aG developers rahul
```

Adds developers without removing existing supplementary group memberships.

But:

```
None
sudo usermod -G developers rahul
```

Replaces Rahul's supplementary group list with developers.

It does not change his primary group.

And:

```
None
sudo usermod -g developers rahul
```

Changes Rahul's primary group.

Important mistake we encountered:

Incorrect:

```
None
sudo usermod -aG developers vishal,rahul
```

This fails because usermod expects one username per invocation.

Correct:

```
None
sudo usermod -aG developers vishal
sudo usermod -aG developers rahul
```

8.4 groups

```
None
groups
```

Displays groups of the current user.

```
None
groups rahul
```

Displays the groups configured for Rahul.

8.5 id

```
None
id
```

Displays current user's UID, primary GID and group memberships.

```
None
id rahul
```

Displays the same information for Rahul.

Example:

```
None
uid=1002(rahul) gid=1002(rahul)
groups=1002(rahul),100(users),1003(developers)
```

Meaning:

- UID: User ID.
- GID: Primary Group ID.
- `1002(rahul)`: Rahul's primary group.
- `100(users)`: Supplementary group.
- `1003(developers)`: Supplementary group.

8.6 getent passwd

```
None
getent passwd
```

Displays account information.

To check one user:

```
None
getent passwd rahul
```

Typical format:

```
None
username:x:UID:GID:comment:home_directory:shell
```

Example:

```
None
rahul:x:1002:1002:Rahul:/home/rahul:/bin/bash
```

Meaning:

- username: rahul
- x: Password information is stored separately.
- UID: 1002
- GID: 1002
- Home: /home/rahul
- Shell: /bin/bash

8.7 getent group

```
None
getent group
```

Displays group information.

To check a particular group:

```
None
getent group developers
```

Example:

```
None
developers:x:1003:vishal,rahul
```

Meaning:

- Group name: developers
- x: Placeholder for group password information
- GID: 1003
- Members: vishal and rahul

8.8 sudo

Allows an authorized user to execute commands with elevated privileges.

Example:

```
None
sudo whoami
```

Output:

```
None
root
```

This does not permanently switch your account to root. It executes that command with elevated privileges.

8.9 su

Switch user.

```
None
su - rahul
```

Switches to Rahul's login shell.

```
None
su - vishal
```

Switches to Vishal's login shell.

---

9. chown and chgrp – Ownership

These commands change file ownership.

9.1 chown

Full form: Change Owner.

Syntax:

```
None
sudo chown username filename
```

Example:

```
None
sudo chown rahul backend.txt
```

Changes the owner to Rahul.

Change owner and group together

```
None
sudo chown vishal:developers backend.txt
```

Here:

- vishal = owner
- developers = group

Recursive ownership change

```
None
sudo chown -R vishal:developers shared-project
```

`-R` means recursively apply the change to the directory and its contents.

9.2 chgrp

Full form: Change Group.

```
None
sudo chgrp developers backend.txt
```

Changes the group ownership without changing the owner.

Important difference

Command	What it changes
chmod	Permissions
chown	Owner and optionally group
chgrp	Group ownership

Changing ownership does not automatically change the file's permission bits or content.

---

10. Our Shared Project Practical

This was one of the most important practical exercises of Day 1.

Our goal:

Allow Vishal and Rahul to work inside the same directory.

Step 1: Create the directory

```
None
mkdir shared-project
```

Step 2: Change ownership

```
None
sudo chown vishal:developers shared-project
```

Now:

- Owner = Vishal
- Group = developers

Step 3: Set permissions

```
None
chmod 770 shared-project
```

Meaning:

- Owner: rwx

- Group: rwx
- Others: ---

Step 4: Enable traversal through Vishal's home directory

```
None
sudo chmod o+x /home/vishal
```

This allowed Rahul to traverse the parent directory.

Step 5: Enable setgid

```
None
chmod 2770 shared-project
```

Now directory permissions:

```
None
drwxrws---
```

Understanding 2770

- First digit `2`: Setgid bit.
- Second digit `7`: Owner permissions = rwx.
- Third digit `7`: Group permissions = rwx.
- Fourth digit `0`: Others = no permissions.

Step 6: Check directory

```
None
ls -ld shared-project
```

Expected format:

```
None
drwxrws--- 2 vishal developers ... shared-project
```

Step 7: Rahul creates a file

Switch to Rahul:

```
None
su - rahul
```

Enter shared directory:

```
None
cd /home/vishal/shared-project
```

Create a file:

```
None
touch frontend.txt
```

Check:

```
None
ls -l
```

The file was owned by:

```
None
rahul developers
```

Why?

- Rahul created the file, so Rahul became its owner.
- The parent directory has setgid, so the file inherited the developers group.

Step 8: Rahul modifies Vishal's file

Vishal created:

```
None
backend.txt
```

Its permissions were:

```
None
-rw-rw-r--
```

Since Rahul belongs to developers, he could append content:

```
None
echo "This line is written by Rahul" >> backend.txt
```

This worked because the group had write permission.

Final understanding

The creator determines the owner.

The setgid parent directory determines the inherited group.

The permission bits determine what other users can do.

---

11. setgid – Special Directory Permission

Setgid stands for Set Group ID.

When setgid is enabled on a directory, newly created files and subdirectories normally inherit the directory's group ownership.

Enable setgid

```
None
chmod 2770 shared-project
```

Check it

```
None
ls -ld shared-project
```

Output:

```
None
drwxrws---
```

Notice the `s` in the group execute position.

This indicates setgid is enabled.

Remove setgid

```
None
chmod 0770 shared-project
```

Output:

```
None
drwxrwx---
```

Now the setgid bit is removed.

Important distinction

Setgid does not automatically give group members write permission.

For example:

```
None
-rw-r--r--
```

Even if the group is developers, other developers cannot write this file.

Setgid controls inherited group ownership, not the actual write permission.

---

12. umask – Default Permission Mask

This was our final topic of Day 1.

Umask determines which permissions are removed when new files and directories are created.

Why do we need umask?

Linux does not normally create every file with full permissions.

It starts with default permissions and removes permissions according to umask.

Default starting permissions

Object	Starting permissions

Regular file	666 = rw-rw-rw-
Directory	777 = rwxrwxrwx

Important: Regular files do not get execute permission by default.

12.1 Check current umask

```
None
umask
```

Example output:

```
None
0002
```

12.2 Symbolic umask

```
None
umask -S
```

Example:

```
None
u=rwx,g=rwx,o=rx
```

This shows the effective permissions allowed by the current umask.

12.3 Change umask

```
None
umask 0022
```

Changes the umask for the current shell session.

It is not automatically a permanent system-wide configuration.

---

13. Our umask Practical

We tested two different umask values.

Case 1: umask 0002

```
None
umask 0002
```

Create a file:

```
None
touch umask-test.txt
```

Create a directory:

```
None
mkdir umask-test-dir
```

Check:

```
None
ls -ld umask-test.txt umask-test-dir
```

Output:

```
None
-rw-rw-r-- ... umask-test.txt
drwxrwsr-x ... umask-test-dir
```

Calculation:

File:

```
None
666 - 002 = 664
```

Directory:

```
None
777 - 002 = 775
```

The directory also shows setgid because its parent directory has setgid enabled.

Case 2: umask 0022

```
None
umask 0022
```

Create a file:

```
None
touch test-0022.txt
```

Create a directory:

```
None
mkdir test-0022-dir
```

Check:

```
None
ls -ld test-0022.txt test-0022-dir
```

Output:

```
None
-rw-r--r-- ... test-0022.txt
drwxr-sr-x ... test-0022-dir
```

Calculation:

File:

```
None
666 - 022 = 644
```

Directory:

```
None
777 - 022 = 755
```

The subtraction is a convenient shorthand for understanding the resulting permissions. Technically, umask works as a permission bitmask that removes specified bits.

Comparison

Umask	New file	New directory
0002	664	775
0022	644	755

Why is 0002 useful for shared projects?

With umask 0002:

```
None
-rw-rw-r--
```

The group gets write permission.

With umask 0022:

```
None
-rw-r--r--
```

The group gets only read permission.

For a shared development directory, 0002 is often more suitable when group members need to modify each other's files.

---

14. Important Practical Questions and Answers

Q1. What is the difference between / and ~?

Answer:

- `/` is the root of the entire Linux filesystem.
- `~` is the home directory of the current user.

Q2. What does cd .. do?

Answer:

Moves one directory level up to the parent directory.

Q3. What is the difference between ls -la and ls -laR?

Answer:

- `ls -la`: Shows hidden files in long format.
- `ls -laR`: Does the same recursively inside subdirectories.

Q4. What is the difference between > and >>?

Answer:

- `>` overwrites existing file content.
- `>>` appends content.

Q5. Does cp remove the original file?

Answer:

No. It creates a copy and keeps the original.

Q6. What is the difference between cp and mv?

Answer:

- `cp` copies.
- `mv` moves or renames.

Q7. What is the difference between rm and rmdir?

Answer:

- `rm` removes files.
- `rmdir` removes empty directories.

Q8. What does 755 mean?

Answer:

- Owner = 7 = rwx
- Group = 5 = r-x
- Others = 5 = r-x

Q9. What does 644 mean?

Answer:

- Owner = rw-
- Group = r--
- Others = r--

Q10. What is the difference between chmod, chown and chgrp?

Answer:

- chmod changes permissions.
- chown changes owner and optionally group.
- chgrp changes group ownership.

Q11. Can a user change the permission of their own file even if they do not have read or write permission?

Answer:

Yes. A file owner can generally change its permission bits using chmod.

Q12. Can someone enter a directory if they have read permission but no execute permission?

Answer:

No. Directory execute permission is required to traverse/search it.

Q13. Can multiple users belong to the same group?

Answer:

Yes. Multiple users can share one group.

Q14. What is the difference between primary and supplementary groups?

Answer:

- Primary group: The user's main group.
- Supplementary groups: Additional groups to which the user belongs.

Q15. What does usermod -aG do?

Answer:

Adds a user to supplementary groups without replacing existing supplementary memberships.

Q16. What is the purpose of setgid on a directory?

Answer:

New files and directories created inside it inherit the directory's group ownership.

Q17. If Rahul creates a file in shared-project, who owns it?

Answer:

Rahul owns it because he created it. Its group becomes developers because of the parent directory's setgid bit.

Q18. Does setgid automatically give group write permission?

Answer:

No. Setgid controls group inheritance, not write permission.

Q19. What does umask control?

Answer:

It controls which permissions are removed from the default permissions of newly created files and directories.

Q20. What is the difference between umask 0002 and 0022?

Answer:

Umask	File	Directory
0002	664	775
0022	644	755

Q21. Can Vishal modify Rahul's file if Vishal belongs to the developers group but the file has 640 permissions?

Answer:

No.

640 means:

- Owner: rw-
- Group: r--
- Others: ---

Vishal can read but cannot write.

Q22. Is umask 0002 permanent?

Answer:

No. Running it in a terminal changes the mask for that shell session. Persistent configuration requires setting it in the appropriate shell or login configuration.

---

15. Common Mistakes We Encountered

Mistake	Correct understanding
`echo "Hello" file.txt`	Does not write into the file
`echo "Hello" > file.txt`	Writes and overwrites
`echo "Hello" >> file.txt`	Appends
`ls -la`	Does not recurse
`ls -laR`	Recursive listing

`chmod g+rw o-r file`	Incorrect syntax for multiple clauses
`chmod g+rw,o-r file`	Correct syntax
`usermod -aG developers vishal,rahul`	Does not accept multiple usernames this way
`chmod 770 shared-project`	Does not automatically enable setgid
`chmod 2770 shared-project`	Enables setgid
`umask 0002`	Does not permanently change every user's umask
`777 - 022` for a regular file	Incorrect starting permissions; regular files start at 666

---

16. Commands Cheat Sheet – Day 1

Command	Purpose
`pwd`	Show current directory
`whoami`	Show current username
`users`	Show logged-in users
`ls`	List files
`ls -l`	Detailed listing
`ls -a`	Show hidden files
`ls -la`	Hidden + detailed listing
`ls -R`	Recursive listing
`cd`	Change directory
`cd ..`	Go to parent directory
`cd ~`	Go to home
`cd /`	Go to filesystem root
`mkdir`	Create directory

`mkdir -p`	Create nested directories
`touch`	Create empty file/update timestamp
`cat`	Display file content
`echo`	Print text
`cp`	Copy files
`cp -r`	Copy directories recursively
`mv`	Move/rename
`rm`	Delete file
`rm -r`	Delete directory recursively
`rm -f`	Force removal
`rmdir`	Remove empty directory
`chmod`	Change permissions
`chown`	Change ownership
`chgrp`	Change group
`adduser`	Create user
`groupadd`	Create group
`usermod`	Modify user
`groups`	Show group membership
`id`	Show UID/GID/groups
`getent passwd`	Display user account information
`getent group`	Display group information
`sudo`	Execute with elevated privileges
`su -`	Switch user with login environment

`umask`	Display/change permission mask

---

17. Final Revision – Five Golden Concepts

If you remember these five concepts, you will have a strong foundation.

1. Linux permissions

Determine who can read, write or execute files and access directories.

2. Ownership

Every file has an owner and a group.

3. Groups

Groups allow multiple users to share access to resources.

4. Setgid

A directory with setgid enabled passes its group ownership to newly created files and directories.

5. Umask

Controls the permissions removed when new files and directories are created.

Day 1 Completion

You have now covered:

- Linux filesystem navigation
- File and directory operations
- Linux users and groups
- File and directory permissions
- Numeric and symbolic chmod
- Ownership management
- Shared directory configuration
- Setgid
- Umask

Next session – Day 2: Linux Process Management

Planned topics:

1. What is a program vs a process?
2. Foreground and background processes.
3. `ps`, `ps aux`, `top`, `htop`.
4. Process IDs (PID) and parent process IDs (PPID).
5. `kill`, `kill -9`, signals.
6. `jobs`, `bg`, `fg`.
7. Running and managing a Node.js backend server.

End of Day 1 notes.